Sketchpad: Rethink from Scratch

Discussion, 1 November 2026. Nothing here is decided or built. "Section 6" means the staged rethink in `docs/NOT_DONE.md` *(also* `docs/Architecture-Rethink.pdf`*), on branch* `claude/batch-1`*.*

Section 6 kept today's shape and cleaned it up. This section starts over. Nothing here is decided or built. Parts are numbered R1 to R10.

The idea is to split Sketchpad into three layers that never mix:

1. **Kernel**: a small, fixed body of geometry code.
2. **Generators**: code that turns parameters into geometry, loaded from a folder.
3. **Documents**: pure JSON describing what was made from generators.

Draw registry objects and scripts both become generators. Figures, Draw settings and composed patterns all become documents.

R1. The kernel: small, fixed, no state

The kernel is the vocabulary everything else uses:

- **Shapes**: point, segment, arc, polyline, sampled curve.
- **Arms**: a path with evenly spaced stops (today's infrastructure).
- **Stitch operations**: parabola between two arms; connect a point set pairwise, as a ring, or all-to-all (Linked Circles); mystic rose; cycloid connection.
- **Services**: layer transforms, rendering to canvas or SVG, hit testing, bounds.

Every function takes values and returns values. There is no `ctx`, no `drawState` and no window globals. Perhaps 2,000 lines, read once. Today that code is spread across `drawUtilities.js`, `drawRegular.js`, `classes.js` and `drawSpirals.js`.

R2. Generators replace the draw registry and the scripts

A generator is one file in a library folder. Everything about it is in that file:

```
export const meta = {
  id: "parabola", version: 3, title: "Parabola",
  tags: ["stitch"], notes: "Two arms stitched at equal steps."
};

export const params = {                // the schema; controls are built from it
  a:     { type: "point", handle: true, default: [100, 300] },
  b:     { type: "point", handle: true, default: [300, 300] },  // the corner
  c:     { type: "point", handle: true, default: [300, 100] },
  steps: { type: "int", min: 3, max: 64, default: 18 },
  color: { type: "color", default: "#0044cc" }
};

export function build(p, k) {          // k = the kernel
  const arm1 = k.arm("arm1", p.a, p.b, p.steps);
  const arm2 = k.arm("arm2", p.b, p.c, p.steps);
  return k.group({ arms: [arm1, arm2],
                   stitches: [k.parabola(arm1, arm2, { color: p.color })],
                   points: { corner: p.b } });
}
```

What this replaces:

- **The `controls` block and the calls to `buildParameterControls()`.** The schema says what kind each param is, so the controls are generated from it.
- **`points[]`, `update()` and `init()`.** A param marked `handle: true` is draggable. Dragging writes that param, and `build()` runs again.
- **The duplicated corner.** The Parabola corner is one param, `b`. Corner tying and `cornerPartners()` are not needed.
- **Magic points (8b).** They are the `points` a generator returns.
- **The window globals.** No `window.drawRegistry_line`, no `runPattern()` with no arguments, no `scriptInfo.redrawHandler`.

Unusual handles. Circular Parabola (Multi)'s second handle separates and joins four copies. A generator can supply its own `drag(name, newPos, params)` that returns new params, so that behaviour stays in the object's own code. The format must not force every object to be simple (the second purpose).

Versions. Registry names stay permanent, and each document records the generator version it used. When a generator's params change, it supplies `migrate(oldVersion, params)`, so old documents still open.

R3. Adding scripts while the app runs

Drop a `.js` file into the library folder, or import one from the app. The Node service notices it and:

1. Checks the shape: a `meta`, a `params` schema and a `build`.
2. Runs it once in isolation with default params.
3. Makes its thumbnail.

4. Adds it to the library index.

The app picks it up without a restart. There are no hand-edited manifests: title, tags and notes live in `meta`, and status lives in a small metadata file beside the script. The index is always rebuilt from the folder, so it cannot drift. That removes `maintain_manifests`, the start-up rebuild, the missing-file check, and hand-written header notes.

R4. Two ways to build a pattern: compose it, or code it

Most pattern scripts are a document written out as code. `OverlappingStars.js` computes 20 named points on a grid, then stitches parabolas between groups of four.

- **Composition, no code.** Place a point-set generator (grid, ring, polygon, spiral), then add stitch layers that refer to its points by name: “parabola from pts 3-7-12, 18 steps”. Change the grid spacing and every stitch follows. Most of `patterns/builds` would become documents.
- **Code, for what is truly procedural.** Parametric curves, roses, pursuit curves and Voronoi stay as generators, written against the kernel.

The rule: compose first; write code only when composition cannot express the pattern.

R5. Analysing foreign scripts

An importer runs any script, Barry’s or downloaded, in a sandbox (a hidden iframe or worker) with a recording canvas. Every `moveTo`, `lineTo` and `arc` is logged as data instead of drawn. One run gives:

- **Geometry as data.** The output can be placed into a figure as a frozen layer (the “flattened earlier drawing” template already decided on). Any script is usable on day one, unconverted.
- **Stitch detection.** Runs of segments whose ends step evenly along two lines are almost certainly parabola stitches. The importer reports, for example: “18-step stitch between (50,150)-(130,150) and (130,150)-(130,250)”. That is the first draft of a document or generator.
- **Parameter candidates.** From a scan of the source: top-level number literals such as `xDist = 80`, offered as sliders.
- **A behaviour report**: globals read or written (`drawState.pts`, `window.*`), which catches the script state leak; network fetches (Voronoi); animation (`requestAnimationFrame`); errors; the cost of a run.
- **A draft header note and a thumbnail**, written automatically.

This replaces hand-written header notes and `SCRIPTS_to_investigate.md`, and is the main tool for converting the 195 scripts. Stitch detection is a heuristic: it suggests a conversion and Barry confirms it.

R6. Figures and the draw registry become one thing

- **Everything opened is a document.** Opening a generator gives a one-layer document (today’s Draw tab). Saving it gives a document (today’s “secondary object”).

- **A layer** is `{ generator: "parabola@3", params, transform, role }`, or a reference to another document.
- **Saved figures can nest.** A saved figure can be inserted as a layer of another, so a star built of parabolas can be reused five times, either as a **reference** (edit once, all five change) or a **copy** (independent).
- **Break apart** means expanding a nested document, or turning a generator's stitches into separate layers.

R7. Separating code from declarative JSON

Rules for a document:

- **JSON holds only data**: generator id and version, params, transforms, roles, point references, constraints (welds), style, metadata, and a format version.
- **Nothing derived is stored.** Today's saved figures keep `points[]` mirrors and computed midpoints, so a file can disagree with itself. Derived values are recomputed on load.
- **Code holds no document state.** Generators are pure, and the kernel has no state.
- **The schema is checked** on load and on save. A failure says which field is wrong; a figure is never half-opened.

```
{ "format": 2, "units": "mm",
  "layers": [
    { "id": 1, "use": "grid@1",     "params": { "cols": 5, "rows": 4, "dx": 80, "dy":
100 } },
    { "id": 2, "use": "parabola@3", "params": { "a": "@1.p3", "b": "@1.p7", "c":
"@1.p12", "steps": 18 } },
    { "id": 3, "use": "doc:stars/smallStar", "transform": { "angle": 30, "tx": 200,
"ty": 0 } }
  ],
  "welds": [ ["@2.corner", "@3.tip1"] ],
  "roles": { "1": "scaffold" } }
```

`@1.p3` reads "point p3 of layer 1". Point references, welds, scaffold and nesting are all data, readable in VS Code.

R8. One integrated workspace instead of six tabs

- **Left: the library.** Generators, documents, scripts and gallery images, with thumbnails, search, tags and status. Replaces the category lists in Home, Draw, Patterns, Gallery and Figures.
- **Centre: the canvas.** It always edits a document.
- **Right: the inspector.** The selected layer's params (controls built from the schema); the layer list; points and welds; the importer's report when a script is selected.
- **Top: a command bar** built from the commands table, plus mode (lines or points) and view (edit, final, print preview).
- **Utilities** become commands or a Tools panel. **Help** becomes a Markdown side panel. Ideabook images are library items that can be placed as a background or trace layer.

Pushback. The six tabs separate activities Barry thinks of separately. If wanted, the workspace can show the library filtered by kind (Draw, Patterns, Gallery) as top tabs, with the same canvas and inspector underneath: the familiar map without six separate implementations.

R9. What carries over from section 6

- One state owner, named actions, and undo.
- Welds as constraints, solved on drag, with the dragged side driving.
- Real units (mm or inches) and SVG/PDF output at size, each weld group printed as one nail.
- Layer roles: stitch, guide, scaffold, image.
- Fast geometry tests in Node; browser tests only for UI wiring.
- Kept from today: stable layer numbers, the layer transform, header comments, the local Node service, and both purposes.

R10. How to get there

A from-scratch rewrite usually dies in the gap where the new thing cannot do what the old one did. The importer (R5) closes that gap: run through the recorder, every old script works in the new app from day one as a frozen or traced layer, and conversion happens one script at a time.

1. **A spike first**, about two weeks of work, in a separate folder (`app2/`) that touches nothing existing: the kernel, the generator format, five generators (Line, Parabola, Grid, Mystic Rose, Circular Parabola Multi, the last because its handle is the hard case), the document format with point references and nesting, and the importer run over ten real scripts.
2. If it holds up: the workspace shell, the library, the inspector and saving.
3. Welds as constraints, undo, and units and print (Phase 9).
4. Conversion in batches. The importer drafts; Barry approves.
5. Retire the old app once nothing in use depends on it.

Recommendation: do the spike. It shows whether the generator format survives the odd objects. The importer is useful even if the current app stays, since its stitch detection and behaviour report would help today. If the spike disappoints, the cost is two weeks, and the staged plan in section 6 is still there.